CS 330, Fall 2026, Homework 1
Due Wednesday, September 9, 2026

Homework Guidelines

Collaboration policy If you choose to collaborate on some problems, you are allowed to discuss each problem with at most 3 other students currently enrolled in the class. Before working with others on a problem, you should think about it yourself for at least 45 minutes.

You must write up each problem solution by yourself without assistance, even if you collaborate with others to solve the problem. You must also identify your collaborators. If you did not work with anyone, you should write "Collaborators: none." It is a **violation of this policy to submit a problem solution that you cannot orally explain to an instructor or TA.**

AI Policy. Getting homework solutions from outside sources, including generative AI tools, the Web, or people not involved in the course, is **not allowed**. See syllabus for details.

Typing. Solutions should be typed and submitted as a PDF file on Gradescope. You may use any program you like to type your solutions. LaTeX, or "Latex", is commonly used for technical writing (`overleaf.com` is a free web-based platform for writing in Latex) since it handles math very well. Word, Google Docs, Markdown or other software are also fine.

Content. To receive *full credit* for an algorithms problem, your solution must contain the following parts (unless explicitly stated that some part is not required)

- A description of the algorithm using pseudocode, with clear comments.
- Analysis of the running time of *your* algorithm.
- Proof of correctness.

Your solution will be graded for correctness, efficiency and clarity.

Teaching Note

In this homework we are including these teaching notes. They aren't homework problems. They are just explaining why we are asking these homework questions and what we hope you will get out of them and learn from them.

1. **Runtime Analysis** (10 points). BU's Computer Science department has decided to issue its own currency, called "bucs", which comes in three denominations: 1, 2, and 3-buc coins. Suppose you have three piles of coins: a one-buc coins, b two-buc coins, and c three-buc coins. The question is this: if you want to give somebody n bucs, how many different ways can you do it using the coins you have? All that matters is the number of coins of each type which you use, not the particular coins.
For example, if $a = 3$, $b = 2$, $c = 2$ and $n = 5$, there are 4 different ways you can choose coins:
$3 \times$ (one buc) $+ 1 \times$ (two bucs)
$2 \times$ (one buc) $+ 1 \times$ (three bucs)
$1 \times$ (one buc) $+ 2 \times$ (two bucs)
$1 \times$ (two buc) $+ 1 \times$ (three bucs)
Your task is to write an algorithm `coinChoices` that takes positive integers a, b, c, n as input and outputs the number of ways to choose coins whose total value is n (where a, b, c are the limits on the number of coins of each type). So `coinChoices(3, 2, 2, 5)` should return 4.
   (a) Here is an algorithm, written as Python[1] code, that solves the problem!

```
def coinChoices(a,b,c,n):
    assert type(a) == type(b) == type(c) == type(n) == int
    assert a> 0 and b>0 and c>0 and n>0
    max_i = min(a, n)
    max_j = min(b, n // 2)
    max_k = min(c, n // 3)
    count = 0
    for i in range(max_i+1):
        for j in range(max_j+1):
            for k in range(max_k+1):
                if i + 2 * j + 3 * k == n:
                    count += 1
                    # print ( (i, j, k), "is a way to get value", n)
    return count
```

   Run this Python function on inputs of the form (n, n, n, n) for values of n that are powers of 2 ranging from 16 to 2048. Measure how much time each execution takes. For example, you could use code like this:

```
import time
powersoftwo =  [2 ** x for x in range(4,12)]
print(powersoftwo)
run_times = []
for n in powersoftwo:
    tick = time.perf_counter()
    print("n =", n)
    coinChoices(n, n, n, n)
    tock = time.perf_counter()
    print(tock-tick, "seconds")
    run_times.append(tock-tick)

```

[1] You may use **any programming language** for this assignment. If you choose something other than Python, you will have to reimplement the algorithm in your chosen language.

Plot the running times you get (using Python, Excel, or any other software), using a **logarithmic scale for the axes** (a "log-log" plot[2]). You should see a roughly straight line! Now try comparing the values you got to a function of the form $t = wn^3$—that is, try to find a value w such that the run times you observed are closely matched by $t = wn^3$. (On one laptop, $w = 5 \times 10^{-8}$ worked well, but your hardware will vary.) For example, you could use Python code like this:

```
from matplotlib import pyplot as plt
w = 5 * 10 ** -8
cubes = [w * n ** 3 for n in powersoftwo]
plt.plot(powersoftwo, run_times, "r.-")
plt.plot(powersoftwo, cubes, "b--")
plt.yscale('log')
plt.xscale('log')
plt.xlabel("n")
plt.ylabel("time (s)")
plt.show()

```

For this part, just submit the value w that you got and the resulting plot. If you were not able to find such a w, explain what you think might have gone wrong.

(b) Give an asymptotically faster algorithm for this problem. (See guidelines: include pseudocode, a proof, and a run time analysis.) For the run time analysis, give a bound as a function of n, assuming a, b, c are at most n (your code should be correct for all such choices of a, b, c, as should your running time analysis). Any algorithm with running time $O(n^2)$ will get full credit (better is great). How fast an algorithm can you find?

(c) Test your runtime analysis experimentally by coding up your algorithm and plotting its running time (as you did above), and seeing if you can approximately match the times with a curve of the form $t = wF(n)$ where $F(n)$ is the asymptotic running time you got in the previous part. Include your plot with two curves: the running time of your algorithm, and that of the best function you could find of the form $t = wF(n)$. Include a brief discussion—a few sentences—of how you chose w and any problems you encountered.

[2]If you are not familiar with log plots, skim the Wikipedia article on "Logarithmic Scale".

2. **Running Sum.** You are given an array B with n elements. We are also given an integer r *as part of the input.* (Remember, this means that you cannot treat it as a constant!) Help us design an algorithm to compute an array A such that the following property holds:

$$A[i] = B[i] + B[i+1] + B[i+2] ... + B[i+r-1]$$

for all i that $0 \leq i \leq n - r$.
In this problem we look at two ways of implementing this algorithm, the first being more straightforward, the second being an order of magnitude faster.

(a) (do not hand in) What is the length of array A? (use n and r.)

(b) Complete the missing parts of SlowFun [3] in Algorithm 1, such that the algorithm has $O(nr)$ iterations.

Algorithm 1: SlowFun (B, r)

```
   /* B is an integer array, r is an integer                    */
1  n ← length(B);
2  A ←array of 0s of appropriate length;
3  for i from [      ] to [      ] do
4      for j from [      ] to [      ] do
5          A[i] ← [                ];
6  return A
```

(c) Complete the missing parts of FastFun in Algorithm 3, such that the algorithm has $O(n)$ iterations.
Note that line 3 doesn't add to the time complexity as $r \leq n$.

Algorithm 3: FastFun (B, r)

```
   /* B is an integer array, r is an integer                    */
1  n ← length(B);
2  A ←array of 0s of appropriate length;
3  A[0] ← B[0] + B[1] + ... B[r − 1] /* runtime of this line is O(r) */
4  for i from [      ] to [      ] do
5      A[i] ← [                ];
6  return A
```

(d) It is easy to see that SlowFun correctly computes the values of A, however it's not as obvious for FastFun. Use induction to formally prove that FastFun correctly computes the values $A[i]$ for every i.

[3] There are some comments in the tex file that can help you with formatting if you are using LaTeX.

Teaching Note

For this problem we are making sure you are back in the swing of thinking about code and loops and variables. We also want to emphasize that two different algorithms can solve the same problem but take very different amounts of time! Finally, we want you to practice the skill of arguing why an algorithm works. Why does FastFun get the right answer? How would you explain this to someone you were working with? Why are *you* confident it works?